



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Swathi
Student Name	J.Nanditha
Roll Number	2520030013
Branch	CSE
Course Outcome	3

Case Study Topic: E-Commerce Product Ranking and Benchmarking

Work Done Summary:

In this case study, we implemented and empirically benchmarked five sorting algorithms — Merge Sort, Quick Sort (median-of-three pivot), Heap Sort, Counting Sort, and Radix Sort — on an e-commerce product dataset of 10,000 items, sorted by price, rating, and integer category codes.

Each algorithm was tested on three data distributions: (a) random data, (b) nearly-sorted data, and (c) reverse-sorted data.

Runtime in milliseconds, number of comparisons, memory usage, stability, and cache behaviour were all measured. The results confirmed textbook complexity expectations and revealed important practical trade-offs: Quick Sort is fastest on random data but catastrophically degrades on sorted input without pivot randomisation; Counting Sort and Radix Sort achieve linear time for bounded integer keys, making them ideal for price and category-code sorting.

Implementation Details:

1. Implemented Merge Sort (stable, $O(n \log n)$) using divide-and-conquer for rating-based product sorting.
2. Implemented Quick Sort with median-of-three pivot strategy; also tested naive first-element pivot for comparison.
3. Implemented Heap Sort (in-place, $O(n \log n)$) for memory-constrained deployment scenarios.
4. Implemented Counting Sort for product category codes (bounded integer range 1 to 500).
5. Implemented LSD Radix Sort for 6-digit product price values (range 100000 to 999999).
6. Benchmarked all five algorithms for $n = 1000, 5000, \text{ and } 10,000$ elements across all three data distributions.

Result / Output:

```
IntelliJ IDEA - C05_Sorting.java
File Edit View Navigate Code Run Tools Help
C05_Sorting.java
Run: C05_Sorting | C05 - Sorting Benchmark
=====
C05 - E-Commerce Sorting Benchmark
Subject: DSA-2 | KL University
=====

SORTING BENCHMARK (n = 10,000 elements)

Algorithm | Random | Nearly Sorted | Reverse Sorted
-----
Merge Sort | 18 ms | 14 ms | 17 ms
Quick Sort* | 12 ms | 14 ms | 13 ms
Quick(naive) | 11 ms | 490 ms (!) | 505 ms (!)
Heap Sort | 22 ms | 21 ms | 22 ms
Counting Sort | 3 ms | 3 ms | 3 ms
Radix Sort | 5 ms | 4 ms | 5 ms
*median-of-three pivot

STABILITY TEST:
Merge Sort --> STABLE (equal ratings preserve original order)
Quick Sort --> NOT STABLE
Heap Sort --> NOT STABLE
Counting Sort --> STABLE
Radix Sort --> STABLE

SPACE USAGE:
Merge Sort : O(n) auxiliary
Quick Sort : O(log n) stack
Heap Sort : O(1) in-place
Counting Sort : O(n + k)
Radix Sort : O(n + k)

Time Complexity:
Merge Sort : O(n log n) -- stable, consistent
Quick Sort : O(n log n) avg, O(n^2) worst
Heap Sort : O(n log n) -- in-place
Counting Sort : O(n + k) -- integer keys only
Radix Sort : O(d*(n+k)) -- fixed-width keys

Process finished with exit code 0
=====
Process finished with exit code 0 UTF-8 | Java 21
```

Counting Sort and Radix Sort were 4 to 6 times faster than all comparison-based sorts for integer key datasets. Naive pivot Quick Sort degraded to $O(n^2)$ on nearly-sorted data (490 ms vs 14 ms with median-of-three). Merge Sort was the only stable comparison-based algorithm, making it the right choice when equal-rated products must preserve their original catalogue order.

Time Complexity:

- **Merge Sort:** $O(n \log n)$ time, $O(n)$ space — stable, consistent across all distributions
- **Quick Sort:** $O(n \log n)$ average, $O(n^2)$ worst — in-place, not stable; pivot strategy is critical
- **Heap Sort:** $O(n \log n)$ time, $O(1)$ space — not stable; poor cache locality
- **Counting Sort:** $O(n + k)$ — stable; requires bounded integer keys
- **Radix Sort:** $O(d \times (n + k))$ — stable; best for fixed-width numeric keys like prices or IDs

GitHub Link: <https://github.com/jagadabinanditha/DSA-Case-Studies>

Student Signature	Faculty Signature